CRICBUZZ LIVESTATS

Real-Time Cricket Insights & SQL-Based Analytics

25 SQL Questions with Solutions

Beginner • Intermediate • Advanced

Prepared as a companion report for the Cricbuzz LiveStats project

1. Introduction

This report organizes the 25 SQL practice questions from the supplied Cricbuzz LiveStats project brief and provides a SQL solution for each. The questions cover player profiles, match history, batting and bowling performance, venues, teams, series, partnerships, toss outcomes, and performance trends.

The original question wording and numbering are retained in substance. The SQL is written for SQLite and uses a proposed analytics schema described below. The current project database described during development has only basic matches and player_stats tables, so many queries will need the additional tables and fields listed here before they can execute.

2. Important data and schema note

Do not treat the sample schema below as a description of the database that already exists. It is a recommended schema contract for these solutions. The supplied project brief asks questions requiring historical ball-by-ball or innings-level data, batting positions, strike rates, economy rates, toss decisions, partnerships, fielding, and series metadata. Those results cannot be calculated reliably from only player_name, team, runs, wickets, matches_played, and basic match details.

No output tables or numerical findings are fabricated in this report. Actual results depend on loading suitable, validated cricket data into the database and running each query.

3. Proposed database schema used by the queries

Table	Required columns
players	player_id, full_name, playing_role, batting_style, bowling_style, country
teams	team_id, team_name, country
venues	venue_id, venue_name, city, country, capacity
series	series_id, series_name, host_country, match_type, start_date, planned_matches
matches	match_id, match_description, team1_id, team2_id, venue_id, match_date, match_type, status, winner_team_id, victory_margin, victory_type, toss_winner_team_id, toss_decision, series_id
player_match_stats	match_id, player_id, team_id, innings_no, runs, balls_faced, fours, sixes, wickets, runs_conceded, overs_bowled, batting_average_value, strike_rate, economy_rate, batting_position, catches, stumpings, did_bat, did_bowl
partnerships	match_id, innings_no, batter1_id, batter2_id, batter1_position, batter2_position, partnership_runs

Assumptions: matches.match_date and series.start_date are stored as ISO dates (YYYY-MM-DD); overs_bowled is stored as a numeric overs value such as 4.2, so a production-grade cricket database should preferably store balls bowled as an integer to avoid arithmetic ambiguity. The examples below assume overs_bowled is usable as a numeric value. For batting average, use runs divided by dismissals rather than blindly averaging innings-level averages if dismissal data is available.

SQLite compatibility: date functions and window functions are used where helpful. SQLite does not provide STDDEV_POP by default, so Question 19 computes standard deviation from average squared deviation.

4. Beginner level — Questions 1–8

Question 1 — Beginner

Find all players who represent India. Display their full name, playing role, batting style, and bowling style.

SQL solution (SQLite)

```
SELECT full_name, playing_role, batting_style, bowling_style
FROM players
WHERE country = 'India'
ORDER BY full_name;
```

Explanation: Filters the player list to Indian players and displays the requested profile fields.

Data note: Requires player profile data in players; the current basic player_stats table does not include role or playing styles.

Question 2 — Beginner

Show all cricket matches that were played in the last 30 days. Include the match description, both team names, venue name with city, and the match date. Sort by most recent matches first.

SQL solution (SQLite)

```
SELECT
m.match_description,
t1.team_name AS team1,
t2.team_name AS team2,
v.venue_name || ', ' || v.city AS venue,
m.match_date
FROM matches AS m
JOIN teams AS t1 ON t1.team_id = m.team1_id
JOIN teams AS t2 ON t2.team_id = m.team2_id
LEFT JOIN venues AS v ON v.venue_id = m.venue_id
WHERE date(m.match_date) >= date('now', '-30 days')
AND date(m.match_date) <= date('now')
ORDER BY date(m.match_date) DESC;
```

Explanation: Joins matches to both teams and the venue, then restricts dates to the last 30 days.

Question 3 — Beginner

List the top 10 highest run scorers in ODI cricket. Show player name, total runs scored, batting average, and number of centuries. Display the highest run scorer first.

SQL solution (SQLite)

```
SELECT
p.full_name AS player_name,
SUM(s.runs) AS total_runs,
ROUND(1.0 * SUM(s.runs) / NULLIF(SUM(CASE WHEN s.dismissal_type IS NOT NULL THEN 1 ELSE 0 END), 0), 2) AS batting_average,
SUM(CASE WHEN s.runs >= 100 THEN 1 ELSE 0 END) AS centuries
FROM player_match_stats AS s
JOIN players AS p ON p.player_id = s.player_id
JOIN matches AS m ON m.match_id = s.match_id
WHERE m.match_type = 'ODI'
GROUP BY p.player_id, p.full_name
ORDER BY total_runs DESC
LIMIT 10;
```

Explanation: Aggregates ODI innings by player, counts innings of 100 or more as centuries, and calculates average runs per dismissal.

Data note: For the average to be exact, add dismissal_type (or a dismissed flag) to player_match_stats. A score of 100+ is counted as a century; use a scorecard-level innings field if retirements/not-outs need special handling.

Question 4 — Beginner

Display all cricket venues that have a seating capacity of more than 50,000 spectators. Show venue name, city, country, and capacity. Order by largest capacity first.

SQL solution (SQLite)

```
SELECT venue_name, city, country, capacity
FROM venues
WHERE capacity > 50000
ORDER BY capacity DESC;
```

Explanation: Filters venues by capacity and sorts from largest to smallest.

Question 5 — Beginner

Calculate how many matches each team has won. Show team name and total number of wins. Display teams with most wins first.

SQL solution (SQLite)

```
SELECT
t.team_name,
COUNT(m.match_id) AS total_wins
FROM teams AS t
LEFT JOIN matches AS m ON m.winner_team_id = t.team_id
GROUP BY t.team_id, t.team_name
ORDER BY total_wins DESC, t.team_name;
```

Explanation: Counts matches where each team is recorded as the winner. LEFT JOIN retains teams with zero wins.

Question 6 — Beginner

Count how many players belong to each playing role (like Batsman, Bowler, All-rounder, Wicket-keeper). Show the role and count of players for each role.

SQL solution (SQLite)

```
SELECT playing_role, COUNT(*) AS player_count
FROM players
GROUP BY playing_role
ORDER BY player_count DESC, playing_role;
```

Explanation: Groups player records by role and counts the players in each group.

Question 7 — Beginner

Find the highest individual batting score achieved in each cricket format (Test, ODI, T20I). Display the format and the highest score for that format.

SQL solution (SQLite)

```
SELECT
m.match_type AS cricket_format,
MAX(s.runs) AS highest_individual_score
FROM player_match_stats AS s
JOIN matches AS m ON m.match_id = s.match_id
WHERE m.match_type IN ('Test', 'ODI', 'T20I')
GROUP BY m.match_type
ORDER BY m.match_type;
```

Explanation: Finds the maximum innings score separately for each requested format.

Question 8 — Beginner

Show all cricket series that started in the year 2024. Include series name, host country, match type, start date, and total number of matches planned.

SQL solution (SQLite)

```
SELECT series_name, host_country, match_type, start_date, planned_matches
FROM series
WHERE date(start_date) >= '2024-01-01'
AND date(start_date) < '2025-01-01'
ORDER BY date(start_date), series_name;
```

Explanation: Uses a date range to return series whose start date falls in calendar year 2024.

5. Intermediate level — Questions 9–16

Question 9 — Intermediate

Find all-rounder players who have scored more than 1000 runs AND taken more than 50 wickets in their career. Display player name, total runs, total wickets, and the cricket format.

SQL solution (SQLite)

```
SELECT
p.full_name AS player_name,
SUM(s.runs) AS total_runs,
SUM(s.wickets) AS total_wickets,
m.match_type AS cricket_format
FROM players AS p
JOIN player_match_stats AS s ON s.player_id = p.player_id
JOIN matches AS m ON m.match_id = s.match_id
WHERE lower(p.playing_role) LIKE '%all-rounder%'
GROUP BY p.player_id, p.full_name, m.match_type
HAVING SUM(s.runs) > 1000 AND SUM(s.wickets) > 50
ORDER BY total_runs DESC;
```

Explanation: Groups all-rounder performance by player and format, then applies both career thresholds within each format.

Data note: If the intention is career totals across all formats, remove m.match_type from SELECT and GROUP BY. The original wording can be interpreted either way.

Question 10 — Intermediate

Get details of the last 20 completed matches. Show match description, both team names, winning team, victory margin, victory type (runs/wickets), and venue name. Display most recent matches first.

SQL solution (SQLite)

```
SELECT
m.match_description,
t1.team_name AS team1,
t2.team_name AS team2,
winner.team_name AS winning_team,
m.victory_margin,
m.victory_type,
v.venue_name
FROM matches AS m
JOIN teams AS t1 ON t1.team_id = m.team1_id
JOIN teams AS t2 ON t2.team_id = m.team2_id
LEFT JOIN teams AS winner ON winner.team_id = m.winner_team_id
LEFT JOIN venues AS v ON v.venue_id = m.venue_id
WHERE lower(m.status) = 'completed'
ORDER BY date(m.match_date) DESC
LIMIT 20;
```

Explanation: Returns the 20 most recent completed matches and enriches each match with team and venue names.

Question 11 — Intermediate

Compare each player's performance across different cricket formats. For players who have played at least 2 different formats, show their total runs in Test cricket, ODI cricket, and T20I cricket, along with their overall batting average across all formats.

SQL solution (SQLite)

```
SELECT
p.full_name AS player_name,
SUM(CASE WHEN m.match_type = 'Test' THEN s.runs ELSE 0 END) AS test_runs,
SUM(CASE WHEN m.match_type = 'ODI' THEN s.runs ELSE 0 END) AS odi_runs,
SUM(CASE WHEN m.match_type = 'T20I' THEN s.runs ELSE 0 END) AS t20i_runs,
ROUND(1.0 * SUM(s.runs) /
NULLIF(SUM(CASE WHEN s.dismissal_type IS NOT NULL THEN 1 ELSE 0 END), 0), 2) AS overall_batting_average,
COUNT(DISTINCT m.match_type) AS formats_played
FROM players AS p
JOIN player_match_stats AS s ON s.player_id = p.player_id
JOIN matches AS m ON m.match_id = s.match_id
GROUP BY p.player_id, p.full_name
HAVING COUNT(DISTINCT m.match_type) >= 2
ORDER BY p.full_name;
```

Explanation: Conditional aggregation creates separate run totals for Test, ODI, and T20I. The HAVING clause keeps players with at least two formats.

Data note: Requires dismissal_type or equivalent dismissal data for a mathematically correct overall batting average.

Question 12 — Intermediate

Analyze each international team's performance when playing at home versus playing away. Determine whether each team played at home or away based on whether the venue country matches the team's country. Count wins for each team in both home and away conditions.

SQL solution (SQLite)

```
WITH team_match_rows AS (
SELECT m.match_id, m.winner_team_id, t.team_id, t.team_name,
t.country AS team_country, v.country AS venue_country
FROM matches AS m
JOIN teams AS t ON t.team_id IN (m.team1_id, m.team2_id)
LEFT JOIN venues AS v ON v.venue_id = m.venue_id
)
SELECT
team_name,
CASE WHEN team_country = venue_country THEN 'Home' ELSE 'Away' END AS condition,
SUM(CASE WHEN winner_team_id = team_id THEN 1 ELSE 0 END) AS wins,
COUNT(*) AS matches_played
FROM team_match_rows
GROUP BY team_id, team_name, condition
ORDER BY team_name, condition;
```

Explanation: Creates one row for each participating team in every match, labels the match home/away using country comparison, and counts wins.

Data note: Neutral venues are classified as Away by this simple rule because the question defines home status by country match. A separate Neutral category would require a different rule.

Question 13 — Intermediate

Identify batting partnerships where two consecutive batsmen (batting positions next to each other) scored a combined total of 100 or more runs in the same innings. Show both player names, their combined partnership runs, and which innings it occurred in.

SQL solution (SQLite)

```
SELECT
p1.full_name AS batter_1,
p2.full_name AS batter_2,
pr.partnership_runs,
pr.innings_no
FROM partnerships AS pr
JOIN players AS p1 ON p1.player_id = pr.batter1_id
JOIN players AS p2 ON p2.player_id = pr.batter2_id
WHERE ABS(pr.batter1_position - pr.batter2_position) = 1
AND pr.partnership_runs >= 100
ORDER BY pr.partnership_runs DESC;
```

Explanation: Filters partnership records to adjacent batting positions and partnerships of at least 100 runs.

Question 14 — Intermediate

Examine bowling performance at different venues. For bowlers who have played at least 3 matches at the same venue, calculate their average economy rate, total wickets taken, and number of matches played at each venue. Focus on bowlers who bowled at least 4 overs in each match.

SQL solution (SQLite)

```
SELECT
p.full_name AS bowler,
v.venue_name,
ROUND(AVG(s.economy_rate), 2) AS average_economy_rate,
SUM(s.wickets) AS total_wickets,
COUNT(DISTINCT m.match_id) AS matches_at_venue
FROM player_match_stats AS s
JOIN players AS p ON p.player_id = s.player_id
JOIN matches AS m ON m.match_id = s.match_id
JOIN venues AS v ON v.venue_id = m.venue_id
WHERE s.did_bowl = 1
AND s.overs_bowled >= 4
GROUP BY p.player_id, p.full_name, v.venue_id, v.venue_name
HAVING COUNT(DISTINCT m.match_id) >= 3
ORDER BY average_economy_rate ASC, total_wickets DESC;
```

Explanation: Includes only match performances with at least four overs, then aggregates by bowler and venue and requires three or more matches.

Question 15 — Intermediate

Identify players who perform exceptionally well in close matches. A close match is one decided by less than 50 runs OR less than 5 wickets. For these close matches, calculate each player's average runs scored, total close matches played, and how many of those close matches their team won when they batted.

SQL solution (SQLite)

```
WITH close_matches AS (
SELECT *
FROM matches
WHERE lower(status) = 'completed'
AND ((lower(victory_type) = 'runs' AND victory_margin < 50)
OR (lower(victory_type) = 'wickets' AND victory_margin < 5))
)
SELECT
p.full_name AS player_name,
ROUND(AVG(s.runs), 2) AS average_runs_in_close_matches,
COUNT(DISTINCT cm.match_id) AS close_matches_played,
SUM(CASE WHEN cm.winner_team_id = s.team_id AND s.did_bat = 1 THEN 1 ELSE 0 END)
AS close_matches_won_when_batting
FROM close_matches AS cm
JOIN player_match_stats AS s ON s.match_id = cm.match_id
JOIN players AS p ON p.player_id = s.player_id
GROUP BY p.player_id, p.full_name
ORDER BY average_runs_in_close_matches DESC;
```

Explanation: The CTE identifies close completed matches; the outer query calculates each player's average runs and counts matches where their team won.

Data note: The count is based on player-match rows. If a player did not bat, they may still appear depending on how player_match_stats is populated; filter to did_bat = 1 if the report should include only batting appearances.

Question 16 — Intermediate

Track how players' batting performance changes over different years. For matches since 2020, show each player's average runs per match and average strike rate for each year. Only include players who played at least 5 matches in that year.

SQL solution (SQLite)

```
SELECT
p.full_name AS player_name,
CAST(strftime('%Y', m.match_date) AS INTEGER) AS match_year,
ROUND(AVG(s.runs), 2) AS average_runs_per_match,
ROUND(AVG(s.strike_rate), 2) AS average_strike_rate,
COUNT(DISTINCT m.match_id) AS matches_played
FROM player_match_stats AS s
JOIN players AS p ON p.player_id = s.player_id
JOIN matches AS m ON m.match_id = s.match_id
WHERE date(m.match_date) >= '2020-01-01'
GROUP BY p.player_id, p.full_name, strftime('%Y', m.match_date)
HAVING COUNT(DISTINCT m.match_id) >= 5
ORDER BY player_name, match_year;
```

Explanation: Groups innings data by player and calendar year, calculates average runs and strike rate, and retains player-years with at least five matches.

6. Advanced level — Questions 17–25

Question 17 — Advanced

Investigate whether winning the toss gives teams an advantage in winning matches. Calculate what percentage of matches are won by the team that wins the toss, broken down by their toss decision (choosing to bat first or bowl first).

SQL solution (SQLite)

```
SELECT
m.toss_decision,
COUNT(*) AS matches_with_toss_winner,
SUM(CASE WHEN m.winner_team_id = m.toss_winner_team_id THEN 1 ELSE 0 END) AS toss_winner_match_wins,
ROUND(100.0 * SUM(CASE WHEN m.winner_team_id = m.toss_winner_team_id THEN 1 ELSE 0 END)
/ NULLIF(COUNT(*), 0), 2) AS toss_winner_win_percentage
FROM matches AS m
WHERE lower(m.status) = 'completed'
AND m.toss_winner_team_id IS NOT NULL
GROUP BY m.toss_decision
ORDER BY m.toss_decision;
```

Explanation: Groups completed matches by toss decision and calculates the share where the toss-winning team also won the match.

Question 18 — Advanced

Find the most economical bowlers in limited-overs cricket (ODI and T20 formats). Calculate each bowler's overall economy rate and total wickets taken. Only consider bowlers who have bowled in at least 10 matches and bowled at least 2 overs per match on average.

SQL solution (SQLite)

```
SELECT
p.full_name AS bowler,
ROUND(1.0 * SUM(s.runs_conceded) / NULLIF(SUM(s.balls_bowled), 0) * 6, 2) AS overall_economy_rate,
SUM(s.wickets) AS total_wickets,
COUNT(DISTINCT m.match_id) AS matches_bowled
FROM player_match_stats AS s
JOIN players AS p ON p.player_id = s.player_id
JOIN matches AS m ON m.match_id = s.match_id
WHERE m.match_type IN ('ODI', 'T20', 'T20I')
AND s.did_bowl = 1
GROUP BY p.player_id, p.full_name
HAVING COUNT(DISTINCT m.match_id) >= 10
AND 1.0 * SUM(s.balls_bowled) / COUNT(DISTINCT m.match_id) >= 12
ORDER BY overall_economy_rate ASC, total_wickets DESC;
```

Explanation: Calculates economy as runs conceded per six legal balls, requires ten bowling appearances, and checks for an average of at least 12 balls (two overs) per match.

Data note: This query assumes balls_bowled stores legal balls as an integer. This is safer than calculating economy from an overs value stored as decimal.

Question 19 — Advanced

Determine which batsmen are most consistent in their scoring. Calculate the average runs scored and the standard deviation of runs for each player. Only include players who have faced at least 10 balls per innings and played since 2022. A lower standard deviation indicates more consistent performance.

SQL solution (SQLite)

```
WITH eligible_innings AS (
SELECT s.player_id, s.runs
FROM player_match_stats AS s
JOIN matches AS m ON m.match_id = s.match_id
WHERE s.balls_faced >= 10
AND date(m.match_date) >= '2022-01-01'
), stats AS (
SELECT player_id, AVG(runs) AS avg_runs, AVG(runs * runs) AS avg_square
FROM eligible_innings
GROUP BY player_id
HAVING COUNT(*) >= 1
)
SELECT
p.full_name AS player_name,
COUNT(e.runs) AS eligible_innings,
ROUND(AVG(e.runs), 2) AS average_runs,
ROUND(SQRT(MAX(0, st.avg_square - st.avg_runs * st.avg_runs)), 2) AS run_standard_deviation
FROM eligible_innings AS e
JOIN players AS p ON p.player_id = e.player_id
JOIN stats AS st ON st.player_id = e.player_id
GROUP BY p.player_id, p.full_name, st.avg_runs, st.avg_square
ORDER BY run_standard_deviation ASC, average_runs DESC;
```

Explanation: Filters innings to 10 or more balls since 2022 and uses variance = $E[x^2] – E[x]^2$ before taking the square root.

Data note: SQLite builds may not include SQRT(). If it errors, calculate variance in SQL and take the square root in Python, or enable an SQLite math extension. For population standard deviation, this formula is appropriate for the filtered innings.

Question 20 — Advanced

Analyze how many matches each player has played in different cricket formats and their batting average in each format. Show the count of Test matches, ODI matches, and T20 matches for each player, along with their respective batting averages. Only include players who have played at least 20 total matches across all formats.

SQL solution (SQLite)

```
SELECT
p.full_name AS player_name,
COUNT(DISTINCT CASE WHEN m.match_type = 'Test' THEN m.match_id END) AS test_matches,
ROUND(1.0 * SUM(CASE WHEN m.match_type = 'Test' THEN s.runs ELSE 0 END) /
NULLIF(SUM(CASE WHEN m.match_type = 'Test' AND s.dismissal_type IS NOT NULL THEN 1 ELSE 0 END), 0), 2) AS
test_average,
COUNT(DISTINCT CASE WHEN m.match_type = 'ODI' THEN m.match_id END) AS odi_matches,
ROUND(1.0 * SUM(CASE WHEN m.match_type = 'ODI' THEN s.runs ELSE 0 END) /
NULLIF(SUM(CASE WHEN m.match_type = 'ODI' AND s.dismissal_type IS NOT NULL THEN 1 ELSE 0 END), 0), 2) AS
odi_average,
COUNT(DISTINCT CASE WHEN m.match_type IN ('T20', 'T20I') THEN m.match_id END) AS t20_matches,
ROUND(1.0 * SUM(CASE WHEN m.match_type IN ('T20', 'T20I') THEN s.runs ELSE 0 END) /
NULLIF(SUM(CASE WHEN m.match_type IN ('T20', 'T20I') AND s.dismissal_type IS NOT NULL THEN 1 ELSE 0 END), 0), 2)
AS t20_average,
COUNT(DISTINCT m.match_id) AS total_matches
FROM players AS p
JOIN player_match_stats AS s ON s.player_id = p.player_id
JOIN matches AS m ON m.match_id = s.match_id
GROUP BY p.player_id, p.full_name
HAVING COUNT(DISTINCT m.match_id) >= 20
ORDER BY total_matches DESC;
```

Explanation: Uses conditional aggregation to report match counts and batting averages by format, retaining players with at least 20 distinct matches overall.

Data note: Requires dismissal data for proper batting averages. The match-type values in the database must be standardized.

Question 21 — Advanced

Create a comprehensive performance ranking system for players. Combine batting performance (runs scored, batting average, strike rate), bowling performance (wickets taken, bowling average, economy rate), and fielding performance (catches, stumpings) into a single weighted score. Use the formula in the project brief and rank players in each cricket format.

SQL solution (SQLite)

```
WITH player_format AS (
SELECT
p.player_id, p.full_name, m.match_type,
SUM(s.runs) AS runs_scored,
ROUND(1.0 * SUM(s.runs) / NULLIF(SUM(CASE WHEN s.dismissal_type IS NOT NULL THEN 1 ELSE 0 END), 0), 2) AS
batting_average,
AVG(s.strike_rate) AS strike_rate,
SUM(s.wickets) AS wickets_taken,
AVG(s.bowling_average) AS bowling_average,
AVG(s.economy_rate) AS economy_rate,
SUM(s.catches) AS catches,
SUM(s.stumpings) AS stumpings
FROM players p
JOIN player_match_stats s ON s.player_id = p.player_id
JOIN matches m ON m.match_id = s.match_id
GROUP BY p.player_id, p.full_name, m.match_type
), scored AS (
SELECT *,
(COALESCE(runs_scored, 0) * 0.01)
+ (COALESCE(batting_average, 0) * 0.5)
+ (COALESCE(strike_rate, 0) * 0.3)
+ (COALESCE(wickets_taken, 0) * 2)
+ ((50 - COALESCE(bowling_average, 50)) * 0.5)
+ ((6 - COALESCE(economy_rate, 6)) * 2)
+ (COALESCE(catches, 0) * 3)
+ (COALESCE(stumpings, 0) * 5) AS weighted_score
FROM player_format
)
SELECT
match_type AS cricket_format,
full_name AS player_name,
runs_scored, batting_average, strike_rate,
wickets_taken, bowling_average, economy_rate, catches, stumpings,
ROUND(weighted_score, 2) AS weighted_score,
RANK() OVER (PARTITION BY match_type ORDER BY weighted_score DESC) AS format_rank
FROM scored
ORDER BY cricket_format, format_rank;
```

Explanation: Aggregates each player's statistics per format, applies the brief's weighted scoring formula, and uses RANK() to rank players within each format.

Data note: The supplied brief's printed bowling-points formula has an unmatched parenthesis in its typography. This solution implements the intended expression as wickets×2 + (50–bowling average)×0.5 + (6–economy rate)×2. Confirm that this interpretation matches your evaluator's expectations.

Question 22 — Advanced

Build a head-to-head match prediction analysis between teams. For each pair of teams that have played at least 5 matches against each other in the last 3 years, calculate total matches, wins for each team, average victory margin when each team wins, performance when batting first vs bowling first at different venues, and overall win percentage for each team.

SQL solution (SQLite)

```
WITH recent_pair_matches AS (
SELECT m.*,
CASE WHEN m.team1_id < m.team2_id THEN m.team1_id ELSE m.team2_id END AS team_a_id,
CASE WHEN m.team1_id < m.team2_id THEN m.team2_id ELSE m.team1_id END AS team_b_id
FROM matches m
WHERE lower(m.status) = 'completed'
AND date(m.match_date) >= date('now', '-3 years')
), eligible_pairs AS (
SELECT team_a_id, team_b_id, COUNT(*) AS matches_played
FROM recent_pair_matches
GROUP BY team_a_id, team_b_id
HAVING COUNT(*) >= 5
)
SELECT
ta.team_name AS team_a,
tb.team_name AS team_b,
COUNT(*) AS matches_played,
SUM(CASE WHEN r.winner_team_id = r.team_a_id THEN 1 ELSE 0 END) AS team_a_wins,
SUM(CASE WHEN r.winner_team_id = r.team_b_id THEN 1 ELSE 0 END) AS team_b_wins,
ROUND(AVG(CASE WHEN r.winner_team_id = r.team_a_id THEN r.victory_margin END), 2) AS avg_margin_team_a_wins,
ROUND(AVG(CASE WHEN r.winner_team_id = r.team_b_id THEN r.victory_margin END), 2) AS avg_margin_team_b_wins,
ROUND(100.0 * SUM(CASE WHEN r.winner_team_id = r.team_a_id THEN 1 ELSE 0 END) / COUNT(*), 2) AS
team_a_win_pct,
ROUND(100.0 * SUM(CASE WHEN r.winner_team_id = r.team_b_id THEN 1 ELSE 0 END) / COUNT(*), 2) AS
team_b_win_pct
FROM recent_pair_matches r
JOIN eligible_pairs ep ON ep.team_a_id = r.team_a_id AND ep.team_b_id = r.team_b_id
JOIN teams ta ON ta.team_id = r.team_a_id
JOIN teams tb ON tb.team_id = r.team_b_id
GROUP BY ta.team_name, tb.team_name, r.team_a_id, r.team_b_id
ORDER BY matches_played DESC;
```

Explanation: Finds pairs with at least five completed head-to-head matches in the last three years and summarizes wins, margins, and win percentages.

Data note: The question also requests batting-first versus bowling-first performance at different venues. That extra split is not included in this compact summary query; it can be added by grouping on venue and toss/innings decision once the database stores the actual batting-first team and venue-level outcomes. Toss decision alone does not always identify which team batted first.

Question 23 — Advanced

Analyze recent player form and momentum. For each player's last 10 batting performances, calculate average runs in their last 5 matches versus their last 10 matches, recent strike-rate trends, number of scores above 50, and a consistency score based on standard deviation. Categorize players as Excellent Form, Good Form, Average Form, or Poor Form.

SQL solution (SQLite)

```
WITH ranked AS (
SELECT
s.player_id, p.full_name, m.match_date, m.match_id,
s.runs, s.strike_rate,
ROW_NUMBER() OVER (
PARTITION BY s.player_id
ORDER BY date(m.match_date) DESC, m.match_id DESC
) AS rn
FROM player_match_stats s
JOIN players p ON p.player_id = s.player_id
JOIN matches m ON m.match_id = s.match_id
WHERE s.did_bat = 1
), last10 AS (
SELECT * FROM ranked WHERE rn <= 10
), summary AS (
SELECT
player_id, full_name,
AVG(CASE WHEN rn <= 5 THEN runs END) AS avg_last5,
AVG(runs) AS avg_last10,
AVG(CASE WHEN rn <= 5 THEN strike_rate END) AS sr_last5,
AVG(CASE WHEN rn BETWEEN 6 AND 10 THEN strike_rate END) AS sr_previous5,
SUM(CASE WHEN runs > 50 THEN 1 ELSE 0 END) AS scores_above_50,
AVG(runs * runs) - AVG(runs) * AVG(runs) AS run_variance,
COUNT(*) AS innings_count
FROM last10
GROUP BY player_id, full_name
)
SELECT *,
ROUND(SQRT(MAX(0, run_variance)), 2) AS run_standard_deviation,
CASE
WHEN avg_last5 >= avg_last10 AND sr_last5 >= COALESCE(sr_previous5, sr_last5)
AND scores_above_50 >= 2 THEN 'Excellent Form'
WHEN avg_last5 >= avg_last10 THEN 'Good Form'
WHEN avg_last5 >= avg_last10 * 0.75 THEN 'Average Form'
ELSE 'Poor Form'
END AS form_category
FROM summary
ORDER BY avg_last5 DESC;
```

Explanation: Uses ROW_NUMBER() to select each player's ten most recent batting performances, compares the most recent five to the previous five, and computes a run consistency measure.

Data note: The project brief does not define exact thresholds for the four form labels. The CASE expression is a proposed rule, not a rule supplied by the source. SQRT() may require Python-side square root if the local SQLite build lacks math functions.

Question 24 — Advanced

Study successful batting partnerships. For pairs of players who have batted together as consecutive batsmen in at least 5 partnerships, calculate average partnership runs, count partnerships exceeding 50, highest partnership score, and success rate. Rank the most successful batting partnerships.

SQL solution (SQLite)

```
SELECT
p1.full_name AS batter_1,
p2.full_name AS batter_2,
COUNT(*) AS partnerships_count,
ROUND(AVG(pr.partnership_runs), 2) AS average_partnership_runs,
SUM(CASE WHEN pr.partnership_runs > 50 THEN 1 ELSE 0 END) AS partnerships_over_50,
MAX(pr.partnership_runs) AS highest_partnership,
ROUND(100.0 * SUM(CASE WHEN pr.partnership_runs > 50 THEN 1 ELSE 0 END) / COUNT(*), 2) AS success_rate_pct
FROM partnerships pr
JOIN players p1 ON p1.player_id = pr.batter1_id
JOIN players p2 ON p2.player_id = pr.batter2_id
WHERE ABS(pr.batter1_position - pr.batter2_position) = 1
GROUP BY p1.player_id, p1.full_name, p2.player_id, p2.full_name
HAVING COUNT(*) >= 5
ORDER BY success_rate_pct DESC, average_partnership_runs DESC;
```

Explanation: Groups consecutive-batter partnerships by player pair, calculates the requested metrics, and keeps pairs with at least five partnerships.

Question 25 — Advanced

Perform a time-series analysis of player performance evolution. Calculate quarterly averages for runs and strike rate, compare each quarter with the previous quarter, identify improving/declining/stable performance, and categorize each player's career phase. Only analyze players with data spanning at least 6 quarters and a minimum of 3 matches per quarter.

SQL solution (SQLite)

```
WITH quarterly AS (
SELECT
s.player_id,
p.full_name,
strftime('%Y', m.match_date) || '-Q' ||
CAST(((CAST(strftime('%m', m.match_date) AS INTEGER) - 1) / 3) + 1 AS INTEGER) AS year_quarter,
(CAST(strftime('%Y', m.match_date) AS INTEGER) * 4
+ CAST((CAST(strftime('%m', m.match_date) AS INTEGER) - 1) / 3 AS INTEGER)) AS quarter_index,
AVG(s.runs) AS avg_runs,
AVG(s.strike_rate) AS avg_strike_rate,
COUNT(DISTINCT m.match_id) AS matches_in_quarter
FROM player_match_stats s
JOIN players p ON p.player_id = s.player_id
JOIN matches m ON m.match_id = s.match_id
GROUP BY s.player_id, p.full_name, year_quarter, quarter_index
HAVING COUNT(DISTINCT m.match_id) >= 3
), compared AS (
SELECT *,
LAG(avg_runs) OVER (PARTITION BY player_id ORDER BY quarter_index) AS previous_quarter_avg_runs,
LAG(avg_strike_rate) OVER (PARTITION BY player_id ORDER BY quarter_index) AS previous_quarter_avg_strike_rate,
COUNT(*) OVER (PARTITION BY player_id) AS eligible_quarters
FROM quarterly
)
SELECT
full_name AS player_name,
year_quarter,
ROUND(avg_runs, 2) AS quarterly_average_runs,
ROUND(avg_strike_rate, 2) AS quarterly_average_strike_rate,
ROUND(previous_quarter_avg_runs, 2) AS previous_quarter_average_runs,
ROUND(previous_quarter_avg_strike_rate, 2) AS previous_quarter_average_strike_rate,
CASE
WHEN previous_quarter_avg_runs IS NULL THEN 'First eligible quarter'
WHEN avg_runs > previous_quarter_avg_runs THEN 'Improving'
WHEN avg_runs < previous_quarter_avg_runs THEN 'Declining'
ELSE 'Stable'
END AS quarter_trend,
CASE
WHEN AVG(avg_runs) OVER (PARTITION BY player_id ORDER BY quarter_index
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
> AVG(avg_runs) OVER (PARTITION BY player_id ORDER BY quarter_index
ROWS BETWEEN 5 PRECEDING AND 3 PRECEDING) THEN 'Career Ascending'
WHEN AVG(avg_runs) OVER (PARTITION BY player_id ORDER BY quarter_index
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
< AVG(avg_runs) OVER (PARTITION BY player_id ORDER BY quarter_index
ROWS BETWEEN 5 PRECEDING AND 3 PRECEDING) THEN 'Career Declining'
ELSE 'Career Stable'
END AS career_phase
FROM compared
WHERE eligible_quarters >= 6
ORDER BY player_name, quarter_index;
```

Explanation: Aggregates player data by quarter, compares each quarter with the previous one using LAG(), and requires at least six eligible quarters after applying the three-match minimum.

Data note: The career-phase CASE compares recent three-quarter averages with the preceding three-quarter averages; this is a transparent proposed interpretation because the brief does not define a numeric trajectory threshold. For robust production analysis, compute and validate this classification over complete quarter series.

7. Implementation checklist

• Confirm and standardize match types (Test, ODI, T20I, and/or T20) in the database.

• Add normalized teams, venues, series, and player profile tables where those fields are missing.

• Store player statistics at match/innings level; include balls faced, dismissals, strike rate, bowling balls, runs conceded, wickets, catches, and stumpings.

• Store match winner, margin, victory type, toss winner, toss decision, and venue country.

• Create partnership records with innings number, both batter IDs, batting positions, and partnership runs.

• Load enough historical data for queries that require minimum match counts, recent ten performances, multiple years, or six quarters.

• Run each query against the actual SQLite schema and save genuine result tables or screenshots for the final report.

• Validate calculated averages and economy rates against scorecards; do not use invented sample outputs.

• If submitting the SQL module inside Streamlit is required, expose these queries in a selectable analytics page after the database schema supports them.

8. Conclusion

The 25 questions demonstrate a progression from basic filtering and aggregation to joins, conditional aggregation, CTEs, window functions, ranking, and time-series analysis. The SQL solutions in this report are a starting point for implementation. Their final correctness depends on matching column names, standardized data values, and the database's actual level of detail. The current project database must be expanded before every query can return meaningful results.

Appendix — Source alignment

Question titles and topic requirements are based on the supplied Cricbuzz.docx project brief: Beginner Questions 1–8, Intermediate Questions 9–16, Advanced Questions 17–25, and the stated expected deliverables. SQL statements, schema assumptions, implementation notes, and suggested thresholds are authored for this report and should be validated against the final project database.